Санкт-Петербургский политехнический университет Петра Великого

Курсовая работа

по дисциплине:

«Объектно-ориентированное программирование»

на тему:

«Система визуализации инерциальной навигации на основе полётного контроллера Pixhawk»

Студент:	Томилов А. С.
Преподаватель:	Ананьевский М. С.
Группа:	3331506/30401

Санкт-Петербург

2026

Contents

Введение **3**

1 Чтение сырых данных с Pixhawk **5**
1.1 Протокол MAVLink и сообщение RAW_IMU 5
1.2 Реализация модуля чтения 5
1.3 Запись данных в CSV 7
1.4 Результаты первичного чтения 7

2 Архитектура системы визуализации **8**
2.1 Обзор проекта 8
2.2 Структура файлов проекта 8
2.3 Межпоточное взаимодействие 9

3 Зависимости и система сборки **10**
3.1 Используемые библиотеки 10
3.2 Конфигурация CMake 10

4 Математика инерциальной навигации **11**
4.1 Калибровка датчиков 11
4.2 Представление ориентации кватернионами 11
4.3 Интегрирование кватерниона 11
4.4 Матрица поворота из кватерниона 12
4.5 Углы Эйлера (конвенция ZYX) 12
4.6 Двойное интегрирование (скорость и позиция) 13
4.7 Метод ZUPT 13

5 Реализация **14**
5.1 Алгоритм интегрирования 14
5.2 Структуры данных 14
5.3 Визуализация 15

6 Инструкция по установке и запуску **16**
6.1 Установка зависимостей 16
6.2 Сборка проекта 16
6.3 Настройка прав доступа к порту 16

6.4 Запуск 16
6.5 Управление в окне визуализации 17

7 Результаты и анализ 18
7.1 Результаты измерения ориентации 18
7.2 Результаты оценки позиции 18
7.3 Анализ причин дрейфа 19
7.4 Методы устранения дрейфа 19

Заключение 21

Список использованных источников 22

Введение

Инерциальная навигация — один из фундаментальных методов определения положения и ориентации объекта в пространстве без использования внешних сигналов [2]. В основе метода лежат данные инерциального измерительного блока (IMU — *Inertial Measurement Unit*), включающего трёхосевые акселерометр и гироскоп. Интегрируя угловые скорости и линейные ускорения во времени, можно восстановить полную навигационную картину: ориентацию, скорость и положение объекта [1].

Полётный контроллер Pixhawk является промышленным стандартом в области автономных летательных аппаратов [10]. Он оснащён высококачественным МЭМС-IMU и поддерживает протокол MAVLink [8] — открытый телеметрический стандарт, позволяющий в реальном времени получать сырые данные датчиков на внешний компьютер [9].

Целью данной работы является разработка программной системы, которая осуществляет чтение сырых IMU-данных (сообщения `RAW_IMU` протокола MAVLink) с полётного контроллера Pixhawk и выполняет их обработку в реальном времени: вычисление ориентации объекта через кватернионное интегрирование гироскопа [3], а также попытку оценки скорости и позиции посредством двойного интегрирования ускорения [6]. Результаты визуализируются в интерактивном 3D-окне с использованием OpenGL и библиотеки GLFW [11].

Задачи работы:

1. реализовать модуль приёма данных по протоколу MAVLink через последовательный порт;

2. реализовать алгоритм калибровки и кватернионного интегрирования гироскопических данных;

3. реализовать алгоритм двойного интегрирования акселерометра для оценки позиции;

4. реализовать механизм ZUPT (*Zero Velocity Update*) для частичной компенсации дрейфа;

5. создать OpenGL-визуализацию навигационного состояния в реальном времени;

6. провести анализ точности полученных результатов.

1 Чтение сырых данных с Pixhawk

1.1 Протокол MAVLink и сообщение RAW_IMU

Первым этапом работы стало освоение взаимодействия с полётным контроллером через протокол MAVLink [8]. MAVLink (*Micro Air Vehicle Link*) — это лёгкий бинарный протокол сериализации для беспилотных систем, разработанный Лоренцем Майером в 2009 году [9]. Протокол широко применяется в экосистеме PX4 и ArduPilot и поддерживается полётным контроллером Pixhawk [10].

Для получения сырых показаний датчиков используется сообщение `RAW_IMU` (идентификатор 27 в диалекте `common`) [9]. Оно содержит девять целочисленных полей в единицах внутренней шкалы датчика (LSB — *Least Significant Bit*):

Table 1: Поля сообщения MAVLink `RAW_IMU`

Поле	**Датчик**	**Описание**
`xacc, yacc, zacc`	Акселерометр	Ускорение по осям X, Y, Z (LSB)
`xgyro, ygyro, zgyro`	Гироскоп	Угловая скорость по осям (LSB)
`xmag, ymag, zmag`	Магнитометр	Магнитное поле по осям (LSB)

1.2 Реализация модуля чтения

Взаимодействие с Pixhawk реализовано через последовательный порт `/dev/ttyACM0` со скоростью 115 200 бод. Инициализация порта выполнена через POSIX-интерфейс `termios` [12], позволяющий настраивать все параметры последовательного интерфейса: скорость, биты данных, паритет, управление потоком.

Ключевые параметры настройки порта: 8 бит данных (CS8), без паритета (~PARENB), один стоп-бит (~CSTOPB), аппаратное управление потоком отключено (~CRTSCTS), таймаут чтения — 1 секунда (`VTIME` = 10).

После инициализации порта запускается основной цикл, в котором байты поступают на вход MAVLink-парсера [9]. Парсер реализован в виде конечного автомата: функция `mavlink_parse_char()` возвращает

ненулевое значение только при получении полного валидного пакета, прошедшего проверку контрольной суммы CRC-16 [8].

```
int setup_serial_port(const char* port, int baudrate) {
    int fd = open(port, O_RDWR | O_NOCTTY);
    struct termios tty;
    memset(&tty, 0, sizeof(tty));
    tcgetattr(fd, &tty);

    cfsetospeed(&tty, baudrate);
    cfsetispeed(&tty, baudrate);

    tty.c_cflag |= (CLOCAL | CREAD);
    tty.c_cflag &= ~CSIZE;
    tty.c_cflag |= CS8;         // 8 data bits
    tty.c_cflag &= ~PARENB;     // no parity
    tty.c_cflag &= ~CSTOPB;     // 1 stop bit
    tty.c_cflag &= ~CRTSCTS;    // no hardware flow control

    tty.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
    tty.c_iflag &= ~(IXON | IXOFF | IXANY);
    tty.c_oflag &= ~OPOST;

    tty.c_cc[VMIN]  = 0;
    tty.c_cc[VTIME] = 10;   // 1 second timeout

    tcsetattr(fd, TCSANOW, &tty);
    return fd;
}
```

Listing 1: Настройка последовательного порта через termios

```
mavlink_message_t msg;
mavlink_status_t  status;
uint8_t buffer[2048];

while (running) {
    int n = read(serial_fd, buffer, sizeof(buffer));
    if (n > 0) {
        for (int i = 0; i < n; i++) {
            if (mavlink_parse_char(MAVLINK_COMM_0,
                                   buffer[i], &msg, &status)) {
                if (msg.msgid == MAVLINK_MSG_ID_RAW_IMU) {
```

```
                        mavlink_raw_imu_t imu;
                        mavlink_msg_raw_imu_decode(&msg, &imu);
                        // store imu.xacc, yacc, zacc, xgyro, ...
                    }
                }
            }
        }
}
```

Listing 2: Основной цикл чтения и парсинга MAVLink

1.3 Запись данных в CSV

Параллельно с обработкой каждый принятый пакет `RAW_IMU` записывается в CSV-файл с меткой времени высокого разрешения, получаемой через `gettimeofday()` с точностью до микросекунды [12]. Принудительный сброс буфера на диск (`flush()`) выполняется каждые 50 пакетов — это компромисс между надёжностью сохранения данных и накладными расходами на операции ввода-вывода.

Формат строки файла:

`timestamp, AX_raw, AY_raw, AZ_raw, GX_raw, GY_raw, GZ_raw, MX_raw, MY_raw, MZ_raw`

1.4 Результаты первичного чтения

В результате данного этапа была подтверждена успешная работа модуля чтения: сообщения `RAW_IMU` стабильно поступали с частотой порядка 100 Гц, что соответствует штатной конфигурации IMU на Pixhawk [10]. Сырые данные акселерометра в состоянии покоя показали устойчивое значение по оси Z (компонента силы тяжести), а гироскоп демонстрировал дрейф нуля, подлежащий компенсации на следующем этапе.

Данные были успешно записаны в CSV-файл для последующего анализа и подбора калибровочных коэффициентов.

2 Архитектура системы визуализации

2.1 Обзор проекта

На основе опыта, полученного при реализации модуля чтения, была разработана полноценная система **Pixhawk IMU Visualizer** — многопоточное приложение на C++, в котором два параллельных потока выполнения взаимодействуют через потокобезопасный разделяемый буфер.

Первый поток реализует чтение данных: он устанавливает соединение с Pixhawk через последовательный порт `/dev/ttyACM0` и непрерывно парсит входящий поток байтов с помощью MAVLink-парсера [9]. При получении полного пакета `RAW_IMU` данные помещаются в очередь и записываются в CSV-файл для последующего анализа.

Второй поток выполняет навигационные вычисления и рендеринг: он извлекает пакеты из очереди, запускает алгоритм интегрирования и обновляет трёхмерную сцену на каждом кадре [11].

2.2 Структура файлов проекта

Table 2: Файлы проекта

Файл	**Назначение**
`src/imu_shared.h`	Общие структуры данных: `RawIMU`, `NavState`, `SharedBuffer`, константы калибровки
`src/imu_visualizer.h`	Объявление функции `run_visualizer()`
`src/imu_visualizer.cpp`	Навигационный алгоритм и OpenGL-рендер
`src/pixhawk_imu_reader.cpp`	MAVLink-ридер, запуск потока визуализации
`CMakeLists.txt`	Система сборки CMake
`c_library_v2/`	Заголовочная библиотека MAVLink C [9]

2.3 Межпоточное взаимодействие

Обмен данными между потоками организован через структуру `SharedBuffer`, включающую:

- `std::deque<RawIMU> queue` — очередь сырых пакетов (максимум 500 элементов) с защитой через `std::mutex`;
- `NavState nav` — текущее навигационное состояние (кватернион, скорость, позиция, углы Эйлера) с отдельным мьютексом;
- `std::deque<array<float,3» trail` — история траектории (до 2000 точек) для трёхмерного отображения;
- `std::atomic<bool> running` — флаг завершения работы, обеспечивающий корректное закрытие обоих потоков.

3 Зависимости и система сборки

3.1 Используемые библиотеки

Table 3: Внешние зависимости проекта

Библиотека	Версия	Роль в проекте
MAVLink `c_library_v2` [9]	2.0	Парсинг телеметрических пакетов, декодирование `RAW_IMU`
GLFW3 [11]	≥3.3	Создание окна OpenGL, ввод мыши и клавиатуры
GLEW	≥2.1	Загрузка расширений OpenGL 3.3
GLM	≥0.9.9	Матричная и кватернионная математика в шейдерах
pthreads	POSIX	Межпоточная синхронизация (`std::thread`)

Библиотека MAVLink является заголовочной — она полностью состоит из `.h`-файлов и не требует отдельной компиляции [8]. Остальные зависимости устанавливаются из репозитория пакетов Ubuntu.

3.2 Конфигурация CMake

Сборочная система использует CMake версии не ниже 3.10. Скрипт `CMakeLists.txt` настроен на стандарт C++14, включает поиск всех зависимостей через `find_package` и `PkgConfig` и транслирует проект с флагами оптимизации `-O2` и предупреждениями `-Wall -Wextra`.

4 Математика инерциальной навигации

4.1 Калибровка датчиков

Сырые показания МЭМС-датчиков содержат постоянное смещение нуля (bias), обусловленное конструктивными особенностями кристалла [1]. Смещения определялись экспериментально по данным датчика в состоянии покоя. Перевод в физические единицы осуществляется умножением на масштабный коэффициент:

$$a_i = \left(\tilde{a}_i - b_{a,i}\right) \cdot S_a, \quad \omega_i = \left(\tilde{\omega}_i - b_{\omega,i}\right) \cdot S_\omega, \tag{1}$$

где $\tilde{a}_i$, $\tilde{\omega}_i$ — сырые целочисленные отсчёты; $b_{a,i}$, $b_{\omega,i}$ — смещения нуля; $S_a = 9{,}81/3050$ [м/с^2/отсчёт], $S_\omega = 0{,}001$ [рад/с/отсчёт].

Числовые значения смещений, использованных в проекте:

$$\mathbf{b}_a = (31,\ 24,\ 0)^\top, \quad \mathbf{b}_\omega = (-7,\ 1{,}5,\ 3)^\top. \tag{2}$$

4.2 Представление ориентации кватернионами

Ориентация тела в пространстве описывается единичным кватернионом [3]:

$$\mathbf{q} = q_w + q_x\,\mathbf{i} + q_y\,\mathbf{j} + q_z\,\mathbf{k}, \quad \|\mathbf{q}\| = 1. \tag{3}$$

Кватернионы предпочтительнее матриц вращения и углов Эйлера по ряду причин [3]: они не страдают от шарнирного замка (*gimbal lock*), компактны (4 параметра против 9) и численно стабильны при нормализации.

4.3 Интегрирование кватерниона

Кинематическое уравнение для кватерниона связывает его производную с угловой скоростью $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$ в системе координат тела [4]:

$$\dot{\mathbf{q}} = \frac{1}{2}\,\mathbf{q} \otimes \begin{pmatrix} 0 \\ \boldsymbol{\omega} \end{pmatrix}, \tag{4}$$

где $\otimes$ — операция умножения кватернионов.

Для численного интегрирования на шаге Δt используется точная формула через ось-угол [1]:

$$\theta = \|\boldsymbol{\omega}\| \cdot \Delta t, \qquad \Delta\mathbf{q} = \left(\cos\frac{\theta}{2},\ \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|}\sin\frac{\theta}{2}\right). \tag{5}$$

Обновление кватерниона и нормализация:

$$\mathbf{q} \leftarrow \frac{\mathbf{q} \otimes \Delta\mathbf{q}}{\|\mathbf{q} \otimes \Delta\mathbf{q}\|}. \tag{6}$$

Нормализация на каждом шаге предотвращает накопление численной ошибки и гарантирует, что $\mathbf{q}$ остаётся единичным [4].

4.4 Матрица поворота из кватерниона

Для перевода векторов из системы координат тела в мировую систему координат используется матрица вращения $\mathbf{R} \in SO(3)$, выраженная через компоненты кватерниона [3]:

$$\mathbf{R} = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{pmatrix}. \tag{7}$$

4.5 Углы Эйлера (конвенция ZYX)

Из кватерниона вычисляются углы крена ϕ, тангажа θ и рыскания ψ по формулам [1]:

$$\phi = \operatorname{atan2}\big(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\big), \tag{8}$$

$$\theta = \arcsin\big(2(q_w q_y - q_z q_x)\big), \tag{9}$$

$$\psi = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big). \tag{10}$$

Функция atan2 обеспечивает однозначный результат в диапазоне $(-\pi, \pi]$, в отличие от arctan.

4.6 Двойное интегрирование (скорость и позиция)

После получения кватерниона ориентации вектор ускорения переводится из системы координат тела в мировую систему и из него вычитается вектор гравитационного ускорения [6]:

$$\mathbf{a}_{\text{world}} = \mathbf{R}\,\mathbf{a}_{\text{body}} - \mathbf{g}_{\text{world}}, \quad \mathbf{g}_{\text{world}} = (0,\ 0,\ -9{,}81)^{\top}\ [\text{м/с}^2]. \tag{11}$$

Последовательным интегрированием методом Эйлера первого порядка вычисляются скорость и позиция [2]:

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \mathbf{a}_{\text{world}}(t) \cdot \Delta t, \tag{12}$$

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{v}(t) \cdot \Delta t. \tag{13}$$

4.7 Метод ZUPT

Ключевой причиной дрейфа позиции при чистом интегрировании является накопление ошибок акселерометра [2]. Метод ZUPT (*Zero Velocity Update*) — классический способ частичной коррекции INS в моменты покоя платформы [7]. Если норма угловой скорости опускается ниже порогового значения, система считает объект покоящимся и мягко обнуляет скорость:

$$\text{если } \|\boldsymbol{\omega}\| < \omega_{\text{th}}: \quad \mathbf{v} \leftarrow \mathbf{v} \cdot \alpha_{\text{damp}}, \tag{14}$$

где $\omega_{\text{th}} = 0{,}015$ рад/с — порог обнаружения покоя, $\alpha_{\text{damp}} = 0{,}85$ — коэффициент затухания. Дополнительно, при выполнении условия ZUPT и норме мирового ускорения ниже 0,3 м/с^2, ускорение обнуляется полностью.

5 Реализация

5.1 Алгоритм интегрирования

Ниже приведён фрагмент, реализующий кватернионное интегрирование в соответствии с формулами (5)–(6):

```
double theta = std::sqrt(gx*gx + gy*gy + gz*gz) * dt;
if (theta > 1e-10) {
    double norm_w = std::sqrt(gx*gx + gy*gy + gz*gz);
    double s  = std::sin(theta * 0.5) / norm_w;
    double c  = std::cos(theta * 0.5);
    double dqw = c, dqx = gx*s, dqy = gy*s, dqz = gz*s;

    // q_new = q (x) dq  (quaternion product)
    double qw = n.qw*dqw - n.qx*dqx - n.qy*dqy - n.qz*dqz;
    double qx = n.qw*dqx + n.qx*dqw + n.qy*dqz - n.qz*dqy;
    double qy = n.qw*dqy - n.qx*dqz + n.qy*dqw + n.qz*dqx;
    double qz = n.qw*dqz + n.qx*dqy - n.qy*dqx + n.qz*dqw;

    double norm = std::sqrt(qw*qw + qx*qx + qy*qy + qz*qz);
    n.qw = qw/norm; n.qx = qx/norm;
    n.qy = qy/norm; n.qz = qz/norm;
}
```

Listing 3: Кватернионное интегрирование гироскопа

5.2 Структуры данных

Все разделяемые данные инкапсулированы в структурах:

```
struct NavState {
    double qw=1, qx=0, qy=0, qz=0;  // quaternion
    double wx=0, wy=0, wz=0;          // angular velocity (rad/s)
    double vx=0, vy=0, vz=0;          // velocity (m/s)
    double px=0, py=0, pz=0;          // position (m)
    double roll=0, pitch=0, yaw=0;    // Euler angles (rad)
    double ax=0, ay=0, az=0;          // acceleration (m/s^2)
};
```

Listing 4: Ключевые структуры данных (imu_shared.h)

5.3 Визуализация

Интерфейс пользователя разделён на две области экрана [11]. В верхней части располагается трёхмерная сцена: модель полётного контроллера, ориентированная в соответствии с текущим кватернионом, траектория движения в виде синей линии с затуханием и мировые оси координат. В нижней части — HUD с четырьмя временными графиками: данные акселерометра, гироскопа, углы Эйлера и текущая скорость.

Управление камерой осуществляется через мышь, сброс траектории — клавишей **R**.

6 Инструкция по установке и запуску

6.1 Установка зависимостей

Все системные зависимости доступны в стандартном репозитории Ubuntu:

```
sudo apt update
sudo apt install build-essential cmake git \
    libglfw3-dev libglew-dev libglm-dev libgl1-mesa-dev
```

Библиотека MAVLink поставляется в виде заголовочных файлов и клонируется непосредственно в каталог проекта [9]:

```
cd pixhawk_imu_visualizer
git clone https://github.com/mavlink/c_library_v2.git
```

6.2 Сборка проекта

```
mkdir build && cd build
cmake ..
make -j$(nproc)
```

При успешной сборке в каталоге `build/` появляется исполняемый файл `pixhawk_imu_reader`.

6.3 Настройка прав доступа к порту

Для обращения к последовательному порту без привилегий суперпользователя необходимо добавить текущего пользователя в группу `dialout` [12]:

```
sudo usermod -a -G dialout $USER
# Re-login required after this command
# Or temporarily (without re-login):
sudo chmod 666 /dev/ttyACM0
```

6.4 Запуск

Перед запуском необходимо подключить Pixhawk к компьютеру через USB. Контроллер определяется системой как устройство `/dev/ttyACM0` [10].

```
cd build
./pixhawk_imu_reader
```

```
# If permission denied on serial port:
sudo ./pixhawk_imu_reader
```

Окно визуализации открывается немедленно при запуске. Данные начинают поступать по мере инициализации Pixhawk. Для завершения программы используется сочетание клавиш Ctrl+C в терминале.

6.5 Управление в окне визуализации

Table 4: Управление в окне визуализации

Действие	**Эффект**
ЛКМ + перетаскивание	Вращение камеры вокруг объекта
Колесо мыши	Приближение / отдаление
Клавиша **R**	Сброс позиции и траектории в $(0, 0, 0)$
Ctrl+C	Завершение программы

7 Результаты и анализ

7.1 Результаты измерения ориентации

В ходе тестирования была подтверждена корректная работа алгоритма кватернионного интегрирования. При вращении полётного контроллера по всем трём осям трёхмерная модель в окне визуализации воспроизводила движения с высокой точностью. Углы Эйлера, вычисленные из кватерниона по формулам (8)–(10), соответствовали физическим поворотам устройства. Данный результат согласуется с известными свойствами алгоритма интегрирования через ось-угол [4].

[Скриншот окна визуализации: трёхмерная модель Pixhawk при повороте на 45° по оси X]

Figure 1: Визуализация ориентации полётного контроллера

7.2 Результаты оценки позиции

Алгоритм двойного интегрирования ускорения показал неудовлетворительные результаты в части оценки реальных координат. Ключевая проблема — **дрейф позиции**: уже через 5–10 секунд движения накопленная ошибка составляла метры, не соответствующие реальному перемещению. Данная проблема является фундаментальной для бескоррекционного инерциального счисления [2].

[График дрейфа позиции: сравнение оценки алгоритма с реальным положением за 30 секунд]

Figure 2: Иллюстрация дрейфа позиции при двойном интегрировании

7.3 Анализ причин дрейфа

Дрейф позиции при чистом инерциальном счислении — известная и фундаментальная проблема [1, 2]. Рассмотрим её математически. Пусть истинное ускорение $a(t)$ и ускорение, измеренное датчиком, $\hat{a}(t)$ связаны:

$$\hat{a}(t) = a(t) + b + \eta(t), \tag{15}$$

где b — постоянное смещение (bias) после калибровки, $\eta(t)$ — случайный шум. Тогда ошибка позиции после двойного интегрирования:

$$\delta p(T) = \frac{1}{2}\, b\, T^2 + \int_0^T (T - t)\, \eta(t)\, dt. \tag{16}$$

Первое слагаемое растёт квадратично по времени даже при малом остаточном bias [6]. Второе слагаемое описывает случайное блуждание. МЭМС-акселерометры класса Pixhawk имеют типичный шум 10^{-3}–10^{-2} м/с^2, что за 10 секунд даёт ошибку позиции порядка единиц метров [2].

7.4 Методы устранения дрейфа

Полноценная инерциальная навигация невозможна без внешних корректирующ сенсоров [6]. Возможные пути решения проблемы:

1. **Фильтр Калмана** с GPS-коррекцией [6] — классический подход в авиационных навигационных системах. GPS предоставляет абсолютную позицию с частотой 1–10 Гц, IMU обеспечивает высокую частоту обновления (100–1000 Гц).

2. **Фильтр Маджвика** [5] — градиентный алгоритм, сочетающий гироскоп, акселерометр и магнитометр. Позволяет существенно улучшить оценку ориентации при медленных движениях, однако не решает проблему позиции.

3. **Нелинейный дополняющий фильтр на SO(3)** [4] — альтернативный подход к оценке ориентации, основанный на теории управления на группах Ли.

4. **Оптический поток** [10] — использование видеопотока с камеры для оценки скорости относительно поверхности. Применяется в системах, где GPS недоступен (помещения).

5. **ZUPT с фильтром Калмана** [7] — расширенное применение метода нулевой скорости в рамках оптимального оценщика состояния.

Заключение

В ходе выполнения данной работы была разработана система визуализации инерциальной навигации на основе полётного контроллера Pixhawk. Реализованы следующие компоненты:

- модуль приёма и парсинга сообщений `RAW_IMU` протокола MAVLink [9] через последовательный интерфейс;
- алгоритм калибровки сырых показаний акселерометра и гироскопа;
- точное кватернионное интегрирование угловой скорости [3] с нормализацие на каждом шаге;
- вычисление матрицы вращения и углов Эйлера из кватерниона;
- алгоритм двойного интегрирования ускорения для оценки позиции [6];
- механизм ZUPT [7] для частичной компенсации дрейфа скорости;
- OpenGL 3.3-визуализация ориентации, траектории и графиков датчиков в реальном времени [11].

Алгоритм оценки ориентации показал высокое качество: повороты по всем трём осям воспроизводились корректно. Алгоритм оценки позиции через двойное интегрирование ускорения продемонстрировал принципиальную проблему накопления дрейфа, характерную для МЭМС-датчиков потребительского класса [1, 2]: ошибка позиции нарастает квадратично по времени и через 5–10 секунд движения достигает метров.

Таким образом, подтверждён известный вывод [6]: получение пригодных для практического использования координат возможно лишь при совместной обработке данных IMU с корректирующими сенсорами в рамках алгоритма фильтра Калмана или аналогичного оптимального оценщика. Данная работа создаёт основу для дальнейшего развития системы в этом направлении.

Список использованных источников

References

[1] Titterton D., Weston J. *Strapdown Inertial Navigation Technology*. 2nd ed. — IET, 2004. — 558 p.

[2] Woodman O. J. *An Introduction to Inertial Navigation*. Technical Report UCAM-CL-TR-696. — University of Cambridge, 2007. — 37 p. — URL: https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf (дата обр. 01.03.2024).

[3] Kuipers J. B. *Quaternions and Rotation Sequences*. — Princeton University Press, 1999. — 398 p.

[4] Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group // *IEEE Transactions on Automatic Control*. — 2008. — Vol. 53, No. 5. — P. 1203–1218. — DOI: 10.1109/TAC.2008.923738.

[5] Madgwick S. O. H., Harrison A. J. L., Vaidyanathan R. Estimation of IMU and MARG orientation using a gradient descent algorithm // *IEEE International Conference on Rehabilitation Robotics (ICORR)*. — 2011. — P. 1–7. — DOI: 10.1109/ICORR.2011.5975346.

[6] Groves P. D. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. 2nd ed. — Artech House, 2013. — 776 p.

[7] Skog I., Handel P. In-Car Positioning and Navigation Technologies — A Survey // *IEEE Transactions on Intelligent Transportation Systems*. — 2009. — Vol. 10, No. 1. — P. 4–21. — DOI: 10.1109/TITS.2008.2011712.

[8] Meier L. et al. MAVLink: A Lightweight Message Marshalling Library for the DroneCode Ecosystem. — URL: https://github.com/mavlink/mavlink (дата обр. 01.03.2024).

[9] MAVLink Developer Guide: Common Messages. — URL: https://mavlink.io/en/messages/common.html (дата обр. 01.03.2024).

[10] PX4 Autopilot User Guide. — URL: https://docs.px4.io/main/en/ (дата обр. 05.03.2024).

[11] GLFW — An OpenGL library. Documentation. — URL: https://www.glfw.org/docs/latest/ (дата обр. 05.03.2024).

[12] The Open Group. The Single UNIX Specification, Version 4. *termios.h — define values for termios.* — URL: https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/termios.h.html (дата обр. 01.03.2024).